

Proyecto #1 — Competencia de Modelación (MLP · Ames House Prices)

CC3092 Deep Learning y Sistemas Inteligentes

Ernesto Ascencio 23009

Repositorio: <https://github.com/AscencioSIUU/deepLearning/tree/main/projects/proy-1-competencia>

1. Análisis exploratorio de datos (EDA)

1.1 Dimensiones y tipos de variables

`train.csv` tiene 1168 filas × 81 columnas. La variable objetivo es `SalePrice` (continua, USD). De las 79 features restantes (se descarta `Id`, identificador sin señal): 36 numéricas (áreas, conteos, años) y 43 categóricas de texto. Dentro de las categóricas, 20 son en realidad **ordinales** con un orden natural (calidad: `Po<Fa<TA<Gd<Ex`; exposición de sótano; terminación de garaje; pendiente del terreno; etc.) y 23 son **nominales puras** sin orden (`Neighborhood`, `SaleType`, `Exterior1st`, ...).

1.2 Estadísticas descriptivas

`SalePrice`: media \$181,442, mediana \$165,000, desviación estándar \$77,264, rango [\$34,900, \$745,000]. Las features numéricas con mayor correlación con el precio son `OverallQual` (calidad general de construcción), `GrLivArea` (área habitable), `GarageCars`/`GarageArea`, `TotalBsmtSF` y `1stFlrSF` — todas relacionadas con tamaño y calidad de la construcción, consistente con la intuición de mercado inmobiliario. `OverallCond` (condición) correlaciona casi nulo, sugiriendo que la calidad de construcción pesa más que el estado de mantenimiento.

1.3 Valores nulos, outliers e inconsistencias

19 columnas tienen nulos. En este dataset, `NA` casi siempre significa “**la casa no tiene esa característica**” (sin piscina, sin garaje, sin sótano) según el diccionario de datos original de Ames — no es un dato faltante real. Tratamiento:

- `PoolQC` (1162 nulos), `MiscFeature` (1122), `Alley` (1094), `Fence` (935), `FireplaceQu` (547), y las columnas `Garage*/Bsmt*/MasVnrType`: nulo → categoría explícita “None”. No se eliminan columnas pese al alto % de nulos, porque el nulo mismo es informativo (ausencia de la característica correlaciona con precio).
- `LotFrontage` (217 nulos, numérica): sin ausencia estructural clara → imputación por mediana (supuesto MCAR).
- `Electrical` (1 nulo): imputación por moda.

Outliers: se identificaron 2 casas con `GrLivArea` > 4000 y `SalePrice` < \$300,000 (`Id` 524 y 1299) — ventas atípicas documentadas del dataset original de Ames que rompen la relación esperada área↔precio. Se eliminan del set de entrenamiento.

1.4 Visualizaciones

Ver docs/figs/: saleprice_dist.png (distribución de SalePrice, sesgo positivo marcado, y su versión log1p mucho más simétrica), outliers_scatter.png (GrLivArea/OverallQual vs. SalePrice, outliers visibles), correlaciones.png (top features numéricas correlacionadas con el precio).

1.5 Decisiones de preprocesamiento (resumen)

Decisión	Justificación
Eliminar Id	identificador, sin señal
Eliminar 2 outliers GrLivArea>4000 & SalePrice<\$300k	ventas atípicas conocidas del dataset
Target $\rightarrow \log_{1p}(\text{SalePrice})$	corrige sesgo positivo; RMSE en log $\approx$ error relativo
Nulos en categóricas de ausencia de feature $\rightarrow$ "None"	el nulo es informativo
LotFrontage $\rightarrow$ mediana; Electrical $\rightarrow$ moda	únicos nulos sin ausencia estructural clara
20 ordinales de calidad $\rightarrow$ OrdinalEncoder con orden real por columna	preserva el orden, evita explotar dimensionalidad
23 nominales puras $\rightarrow$ one-hot	sin orden natural
36 numéricas $\rightarrow$ StandardScaler	MLP con gradiente converge mejor con features en escala similar
+4 features derivadas: TotalSF, HouseAge, RemodAge, TotalBath	bajan el RMSE de CV $\sim 15\%$; el MLP no reconstruye fácilmente "área total" a partir de sus 3 componentes por separado con ~ 930 filas de train

1.6 Feature engineering

Además del preprocesamiento de columnas crudas, se agregaron 4 columnas derivadas (engineer_features en src/preprocessing.py), agregadas **después** de excluir outliers para no derivar de filas que luego se descartan:

Feature derivada	Fórmula	Motivación
TotalSF	TotalBsmtSF + 1stFlrSF + 2ndFlrSF	área total, más correlacionada con precio que cualquier componente por separado
HouseAge	YrSold - YearBuilt	antigüedad, más directamente interpretable que el año de construcción crudo
RemodAge	YrSold - YearRemodAdd	años desde la última remodelación
TotalBath	FullBath + 0.5·HalfBath + BsmtFullBath + 0.5·BsmtHalfBath	conteo total de baños ponderado

Estas 4 features bajaron el RMSE de 5-fold CV de la config ganadora de **0.227 (log) / \$43,278 (USD)** a **0.209 (log) / \$38,068 (USD)** — mejora consistente en los 5 folds, no un artefacto de un fold particular (sección 4.1).

2. Metodología de desarrollo

Arquitecturas consideradas: MLPs totalmente conectados con 2–3 capas ocultas, anchura entre 32 y 256 neuronas por capa, activación ReLU, dropout opcional (0–0.3), BatchNorm1d opcional, salida de una neurona (regresión). Se probaron 8 combinaciones distintas (sección 3).

División de datos: split 80/20 train/validation con seed fija (42) para el sweep de iteraciones, sin fuga de información (el ColumnTransformer se ajusta solo con train). La config ganadora se validó además con **5-fold cross-validation** (cross_validate_config en src/model.py): en cada fold se reajusta el pipeline de preprocesamiento únicamente con los datos de ese fold de train, evitando fuga entre folds, y el modelo se reentrena desde cero.

Modelo final: ensemble de 5-fold CV. En vez de reentrenar un único modelo sobre todo train.csv, train.py guarda los 5 modelos de la validación cruzada como ensemble; predict.py promedia sus predicciones en log-espacio antes de invertir a USD. Medido sobre un holdout del 15% **nunca visto por ningún fold** (split adicional, solo para esta medición honesta), el ensemble bajó el RMSE de \$40,259 (un solo modelo) a **\$33,610** — variance reduction estándar de ensembles, sin entrenamiento extra respecto al CV que de todas formas se corre. predict.py también aplica un clip de seguridad $[0.5 \times \min(\text{SalePrice}), 1.5 \times \max(\text{SalePrice})]$ observado en train, para acotar el daño de una extrapolación catastrófica (sección 4, “fragilidad ante extrapolación”).

Función de pérdida, optimizador, hiperparámetros: MSELoss sobre el target en escala $\log_{10}(\text{SalePrice})$ (evita que las pocas casas caras dominen el gradiente y hace que el error penalice proporcionalmente en toda la escala de precios). Optimizador Adam, learning rate 1e-3 (o 3e-4 en una variante), batch size 32.

Regularización: early stopping por RMSE de validación (paciencia 20–25 épocas, restaura los pesos del mejor punto visto) en todas las configs; dropout, weight decay (L2) y batch normalization se probaron como variantes adicionales (sección 3).

3. Resultados de iteraciones

Sweep de 8 configuraciones, cada una variando **un solo eje** respecto a un baseline común, para poder atribuir el efecto de cada cambio:

Config	Arquitectura	Dropout	Weight decay	BatchNorm	lr	Épocas	Train RMSE (log)	Val RMSE (log)	Val RMSE (USD)
C1_baseline	64→32	0.0	0	No	1e-3	194	0.063	0.204	\$41,401
C2_mas_casas	128→64→32	0.0	0	No	1e-3	93	0.070	0.237	\$73,019
C5_weight_decay	128→64	0.0	1e-4	No	1e-3	62	0.109	0.255	\$89,568
C7_dropout	128→64	0.2	1e-4	No	1e-3	64	0.277	0.267	\$47,996
C8_lr_bajo	128→64	0.2	1e-4	No	3e-4	64	0.267	0.300	\$63,974
C3_mas_ancho	256→128	0.0	0	No	1e-3	66	0.095	0.320	\$340,645
C4_dropout	128→64	0.3	0	No	1e-3	40	0.497	0.328	\$69,205
C6_batchnorm	128→64	0.0	0	Sí	1e-3	49	0.356	0.522	\$175,697

Curvas de entrenamiento de las configs más relevantes: docs/figs/sweep_comparacion.png.

Evidencia de overfitting: C3 (más ancho, sin regularización) tiene el mayor gap train/val entre las configs “limpias” ($0.320 - 0.095 = 0.226$) y un RMSE en USD desproporcionado (\$340,645) — 256→128 sin dropout/weight_decay memoriza patrones espurios de las ~930 filas de entrenamiento que no generalizan.

Evidencia de inestabilidad: C6 (batchnorm) fue la peor config en general. Con batch_size=32 sobre un dataset de este tamaño, las estadísticas de batch que normaliza BatchNorm1d son ruidosas, introduciendo

inestabilidad en vez de acelerar la convergencia — consistente con la recomendación general de usar batch-norm en datasets grandes con batches más estables.

Validación cruzada de la config ganadora: para confirmar que C1 no ganó por ruido del split 80/20, se corrió 5-fold CV reajustando el pipeline en cada fold. Con las columnas crudas: **val RMSE log = 0.227 ± 0.035 , val RMSE USD = $\$43,278 \pm \$9,175$** — consistente con el \$41,401 del split original, confirmando que la ventaja del baseline es robusta y no un artefacto de un único split. Con las 4 features derivadas de la sección 1.6, el mismo CV mejora a **val RMSE log = 0.209 ± 0.020 , val RMSE USD = $\$38,068 \pm \$3,342$** — también reduce la varianza entre folds (desviación estándar USD de \$9,175 a \$3,342), señal de un modelo más consistente entre subconjuntos de datos.

4. Discusión de resultados

Comparación entre iteraciones: el baseline (C1, el modelo más simple del sweep) ganó con margen claro. Con 223 features (tras el one-hot) y ~930 filas de entrenamiento efectivas, la razón parámetros/muestras ya es alta incluso con la arquitectura más chica (64→32 ≈ 16.5k parámetros). Añadir capacidad (C2, C3) sin compensar con regularización solo empeoró el gap train/val. Sorprendentemente, dropout y weight decay (C4, C5, C7) tampoco mejoraron sobre C1 — indicio de que el early stopping por sí solo ya regularizaba lo suficiente, y que el dropout adicional restó capacidad útil antes de que el modelo terminara de aprender la señal (el `train_rmse` de C4 quedó en 0.497, muy por encima de las demás — subentrenado en las pocas épocas que corrió antes del early stop).

Análisis de errores del modelo final: sobre el propio set de entrenamiento (desempeño optimista, no generalización), el RMSE es \$13,037 excluyendo los 2 outliers conocidos. Los residuos tienen media -\$5,497 (el modelo tiende a **subestimar** ligeramente el precio en promedio) y desviación estándar \$11,827. Los errores más grandes concentran casas de valor alto o atípico: una casa en OldTown de \$475,000 se sobreestima en \$122,146; casas pequeñas (958–1795 ft²) en Somerset/NAMES se sobreestiman por \$50k–\$70k. El patrón sugiere que el modelo generaliza peor en los extremos de la distribución de precio/tamaño que en el rango típico (\$130k–\$215k), donde está la mayoría de los datos de entrenamiento.

Fragilidad ante extrapolación (y su mitigación): al validar `predict.py` sobre `train.csv` completo (incluyendo los 2 outliers excluidos del entrenamiento), la casa `Id=1299` (`GrLivArea=5642`, precio real \$160,000) recibía una predicción de **\$3.88M** antes de agregar el clip de seguridad — un error de +\$3.7M en un solo punto. El modelo nunca vio casas con esa área durante entrenamiento; en log-espacio el error es moderado, pero `expm1` lo amplifica exponencialmente al volver a USD. Con el clip `[0.5×min(SalePrice), 1.5×max(SalePrice)]` agregado en `predict.py`, esa misma predicción queda acotada a \$1.12M — sigue siendo un error grande, pero su contribución al RMSE cuadrático baja ~15×. Sigue siendo una limitación real: el clip acota el daño, no lo elimina; si el held-out de competencia contiene casas muy fuera del rango de `train.csv`, el error en esos puntos concretos seguirá siendo alto.

Trade-off complejidad/generalización: el hallazgo central del sweep es que, para este dataset (~1000 filas, 223 features tras codificación), **menos es más:** el modelo más simple generalizó mejor que todas las variantes con más capacidad o más regularización explícita. Esto es consistente con la teoría de aprendizaje estadístico — con pocas muestras relativas a la dimensión, la complejidad efectiva del modelo debe mantenerse baja, y el early stopping ya cumple ese rol sin necesitar mecanismos adicionales.

Limitaciones del enfoque y del dataset: (1) el sweep completo (8 configs) usó un solo split 80/20, con ruido de muestreo no cuantificado en el ranking de configs cercanas — mitigado solo para la ganadora vía CV posterior, no para las 8 configs completas por presupuesto de tiempo; (2) el one-hot de `Neighborhood` (25 categorías) por sí solo aporta ~25 columnas dispersas, con pocas observaciones por categoría en los barrios menos frecuentes; (3) el modelo extrapola mal fuera del rango de entrenamiento, mitigado pero no eliminado por el clip de predicción; (4) el clip de seguridad es un límite fijo por percentil de train, no una calibración de incertidumbre aprendida — un enfoque más robusto (p. ej. un modelo de cuantiles o intervalos de predicción) queda como trabajo futuro.

5. Conclusiones

- **Desempeño final:** el modelo de producción (ensemble de 5-fold CV + feature engineering + clip de seguridad) alcanza **RMSE $\approx$ \$33,610 USD** (0.171 en log), medido sobre un holdout del 15% nunca visto por ningún fold del ensemble — la estimación más honesta disponible de generalización. Esto representa una mejora del **~19%** sobre la primera versión del modelo (config ganadora del sweep, sin feature engineering ni ensemble: \$41,401 USD). El desglose del progreso: \$41,401 (config base) $\rightarrow$ \$38,068 (+ feature engineering, CV por fold) $\rightarrow$ \$33,610 (+ ensemble, medido en holdout independiente).
- **Aprendizajes técnicos:** (1) entrenar sobre el target en escala log es determinante en datasets de precios con sesgo positivo, tanto para la optimización como para la interpretabilidad del error; (2) con pocas muestras relativas a la dimensión, un modelo simple con early stopping puede superar variantes con más capacidad o regularización explícita — la regularización “correcta” no es automáticamente “más regularización”; (3) invertir una transformación no lineal (`expm1`) amplifica errores de forma no uniforme, hay que revisar el error en la escala en que realmente importa (USD), no solo en la escala de entrenamiento (log); (4) validar la config ganadora con k-fold CV es barato (~15s en CPU) y da una estimación de generalización con incertidumbre cuantificada; (5) el ensemble de modelos de k-fold CV es prácticamente gratis — ya se entrenan para la validación cruzada, solo falta guardarlos y promediar sus predicciones — y en este proyecto dio la mayor mejora individual de RMSE; (6) un clip de seguridad simple, aunque no resuelve la extrapolación, es una red de contención barata contra errores catastróficos de un único punto.
- **Mejoras futuras:** extender el 5-fold CV a las 8 configs completas del sweep (no solo a la ganadora) para un ranking más robusto; un modelo de incertidumbre (cuantiles, intervalos de predicción) en vez del clip fijo actual; más feature engineering (interacciones entre `OverallQual` y área, por ejemplo) para seguir bajando el RMSE; ensembles heterogéneos (MLPs con distintas arquitecturas, no solo distintos folds) para mayor diversidad.

6. Enlace al repositorio de GitHub

<https://github.com/AscencioSIUU/deepLearning/tree/main/projects/proy-1-competencia>

Contiene el código completo: EDA y desarrollo (`proy1_mlp.ipynb`, generado por `build_nb.py`), preprocesamiento (`src/preprocessing.py`), modelo, training loop y cross-validation (`src/model.py`), entrenamiento final (`train.py`, serializa el pipeline con el módulo estándar `pickle` y los pesos con `torch.save`) y predicción/evaluación (`predict.py`), con instrucciones de reproducción en el `README.md` del proyecto.